Take-home: behind-the-meter battery dispatch for a Cyprus hotel

2–3 hours, lean on AI the whole way through. A small slice of what we do at Neura: running behind-the-meter batteries for commercial customers in Cyprus.

Quick background (skip if you've worked in energy)

Behind-the-meter (BTM) means the battery sits on the customer's side of the utility meter. It sees the building's solar and load directly; the only thing the grid sees is the net flow in and out. The customer's lever is their own electricity bill, not wholesale prices.

What's the battery for? It lets you time-shift energy. Charge when energy is cheap or free — surplus rooftop solar at midday, off-peak grid overnight. Discharge when energy is expensive — peak grid hours, or to cover load that solar can't reach. The moment-to-moment "charge / discharge / idle" decision is called **dispatch**.

A few terms you'll see below:

- **TOU** (time-of-use) tariff — grid price varies by hour.
- **SoC** — state of charge, i.e. how full the battery is, as a %.
- **Round-trip efficiency** — energy lost across one full charge→discharge cycle. ~88% for modern LFP, so you lose ~12%.
- **Curtailment** — throwing energy away because nothing can absorb it. If solar's producing more than the building can use and the battery is full, the excess is curtailed (Cyprus net-metering for commercial sites is restricted, so you can't just push it to the grid).
- **Self-consumption** — share of solar output that gets used on-site rather than curtailed.

The scenario

A 4-star hotel in Limassol has just commissioned a **200 kWp rooftop PV array** and a **400 kWh / 200 kW lithium iron phosphate battery** — i.e. 400 kWh of capacity, up to 200 kW of charge or discharge. Their peak grid draw on a hot summer afternoon was around **200 kW**.

They want a small internal tool that (1) tells them how much money the battery is saving them, (2) shows the dispatch schedule it's following, and (3) spits out a weekly report they can show their financiers. Build a slice of that as a small Django service (Python 3.11+, SQLite is fine).

What to build

1. Data — three 15-minute series for one representative week

- `solar_kw` — PV production. Pull from renewables.ninja using a Limassol lat/long and a 200 kW system. They give hourly; resample to 15-min and document how.
- `load_kw` — the hotel's demand. No clean public 15-min dataset exists for a Cyprus hotel, so build one. Pick a reasonable daily and weekly shape — occupancy-driven, hot afternoons spike for cooling, weekends differ from weekdays — and scale it so the weekly peak hits ~200 kW. Write the assumptions in the README. (DOE/OpenEI reference building load shapes are a fine sanity reference.)
- `grid_price_eur_per_kwh` — use this stylised 2-rate TOU, modelled on EAC's residential Code 02:
  - Day 09:00–23:00: **€0.30/kWh**
  - Night 23:00–09:00: **€0.15/kWh**

  (Pulling a real EAC commercial tariff is a stretch goal — their PDFs are partly Greek and sometimes geo-blocked from outside Cyprus.)

Store the series in the database. Don't hardcode them in `views.py`.

2. Dispatch — charge / discharge / idle at each 15-min step

Constraints:

- SoC stays between **10% and 95%**.
- Charge and discharge power capped at **200 kW**.
- Round-trip efficiency ~88%.
- No grid export. Surplus solar the battery can't absorb is curtailed.
- Cover load from solar first, then battery, then grid.

A greedy policy is fine — "soak up surplus solar, discharge during the day-rate peak". No LP solver needed.

3. Weekly report — one Django view at `/reports/weekly/`

- Grid spend **with** the battery vs. a **no-battery** counterfactual, with the saving on top.
- kWh charged and discharged over the week.
- Solar self-consumption % (share of PV used on-site vs. curtailed).
- The SoC curve over the week. Any plotting library — matplotlib, Plotly, Chart.js, whatever.

What to send us

- A public GitHub repo. README covers setup, your data sources, your assumptions, and what you'd build next.
- Runs locally with one command after `pip install -r requirements.txt`. We'll actually run it.
- A short Loom (≤ 3 min) covering: one thing in the data that surprised you, one assumption you made and why, and what you'd build next with another day.

What we're looking at

Signal	What good looks like
Finding data	You actually used renewables.ninja. Where you had to invent (the hotel load), you did it defensibly and wrote it down.
Architecture	Sensible Django models. Dispatch lives somewhere that isn't a view. Reports don't reach into the policy.

Signal	What good looks like
Code	Idiomatic Django. Some tests on the dispatch logic. Type hints if you use them.
Judgement	You make calls, document them, flag uncertainty. You don't email us for permission.
How you used AI	You drove Claude / Cursor / Copilot like a senior driving a junior. Tell us which tools and where they helped or got in the way.

Stretch (only if you have the time)

- Pull a real EAC commercial tariff from `eac.com.cy` (PDF, partly Greek, sometimes geo-blocked) and re-run the dispatch against it. Tell us how the saving shifts.
- A small "what-if" form: vary battery size or PV size and see how the weekly saving shifts.

Ground rules

- 2–3 hours. If you blow past, say so in the README.
- Use AI heavily. We want to see what you ship *with* it, not without.
- Anything pip-installable is fine.
- If something's ambiguous, make a sensible call, write it down, keep going.
- An ugly working thing with a clear README beats a polished half-built one.
- If you're done in 60 minutes, ship it. Don't pad.

> **One thing.** This is genuinely the kind of problem we work on. If you enjoyed it, tell us. If you found it dull, also tell us. Honest signal in either direction beats a polished pretence.